ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR

Membre de HONORIS UNITED UNIVERSITIES

Compte Rendu d'Atelier 3

API REST avec FastAPI : le projet LogiAI

Réalisé par:

Boujrada Younes

École : EMSI, Les Orangers

Filière : 5IADO

Groupe : G3

Développement et déploiement d'applications intelligentes

Professeur : soukaina gouraguine

2026/2027

1. Environnement et premiers tests

Sortie de `python -m pytest` au démarrage (état initial) :

```
..\.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\PC\Downloads\logiai_starter\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using
 ng `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================== short test summary info ==========================================

FAILED tests/test_api.py::test_predict_valide_renvoie_200 - assert 422 == 200
FAILED tests/test_api.py::test_predict_forme_de_la_reponse - AssertionError: assert {'detail'} == {'alerte', 'm...ard', 's
euil'}
FAILED tests/test_api.py::test_predict_probabilite_entre_git switch -c tp-apiprobabilite_retard'
>> git add ./test_api.py::test_predict_trajet_risque_plus_eleve_que_trajet_facile - KeyError: 'probabilite_retard'
>> git commit -m "TP API : schema, predict, tests, client Streamlit"2 == 503
>> git push -u origin tp-api========== 5 failed, 8 passed, 1 warning in 3.48s ===================================
```

Performances initiales du modèle (`python -m logiai.train`) :

- **Baseline accuracy** : 0.726 (72.6 % en prédisant toujours « à l'heure »)
- **Accuracy du modèle** : 0.774 (77.4 %)
- **Rappel du modèle** : 0.464 (46.4 %)
- **Précision** : 0.617 (61.7 %) | **AUC** :
- *Interprétation :* Sur 100 livraisons réellement en retard, environ 46 sont détectées par le modèle. Pour un dispatcher, ce modèle apporte une valeur ajoutée immédiate en priorisant l'action proactive sur près de la moitié des retards réels plutôt que de subir passivement les aléas de livraison.

Pourquoi certains tests attendant un code 422 passaient-ils déjà alors que rien n'était encore codé ?

Le schéma `Livraison` était configuré avec `model_config = ConfigDict(extra="forbid")` et n'avait aucun champ déclaré (seul `distance_km` était présent). Dès lors, tout champ transmis dans la requête HTTP était considéré comme inconnu et interdit, provoquant systématiquement un rejet automatique par Pydantic avec une erreur **422 Unprocessable Entity**. Les tests attendant 422 réussissaient donc par « faux vert » car tout était rejeté, y compris les données légitimes.

2. Schéma Pydantic (Livraison)

Définition des 6 variables restantes avec leurs bornes physiques réalistes :

Variable	Min	Max	Justification physique
`distance_km`	1	1000	D'un court trajet urbain (1 km) jusqu'à une liaison nationale longue distance (1000 km).
`poids_kg`	0.1	30000	Du pli/colis léger en moto (100 g) jusqu'à la capacité d'un camion lourd (30 tonnes).
`heure_depart`	0	23	Heure sur le cycle journalier de 24 heures (0 à 23).
`pluie_mm`	0	300	Temps sec (0 mm) jusqu'à des conditions orageuses ou d'inondation extrêmes (300 mm).
`trafic_index`	0	10	Échelle standardisée fixée dans le contrat (0 = parfaitement fluide, 10 = réseau saturé).
`nb_arrets`	0	50	Trajet direct sans arrêt (0) jusqu'à une tournée urbaine intensive de livraison (50 arrêts).

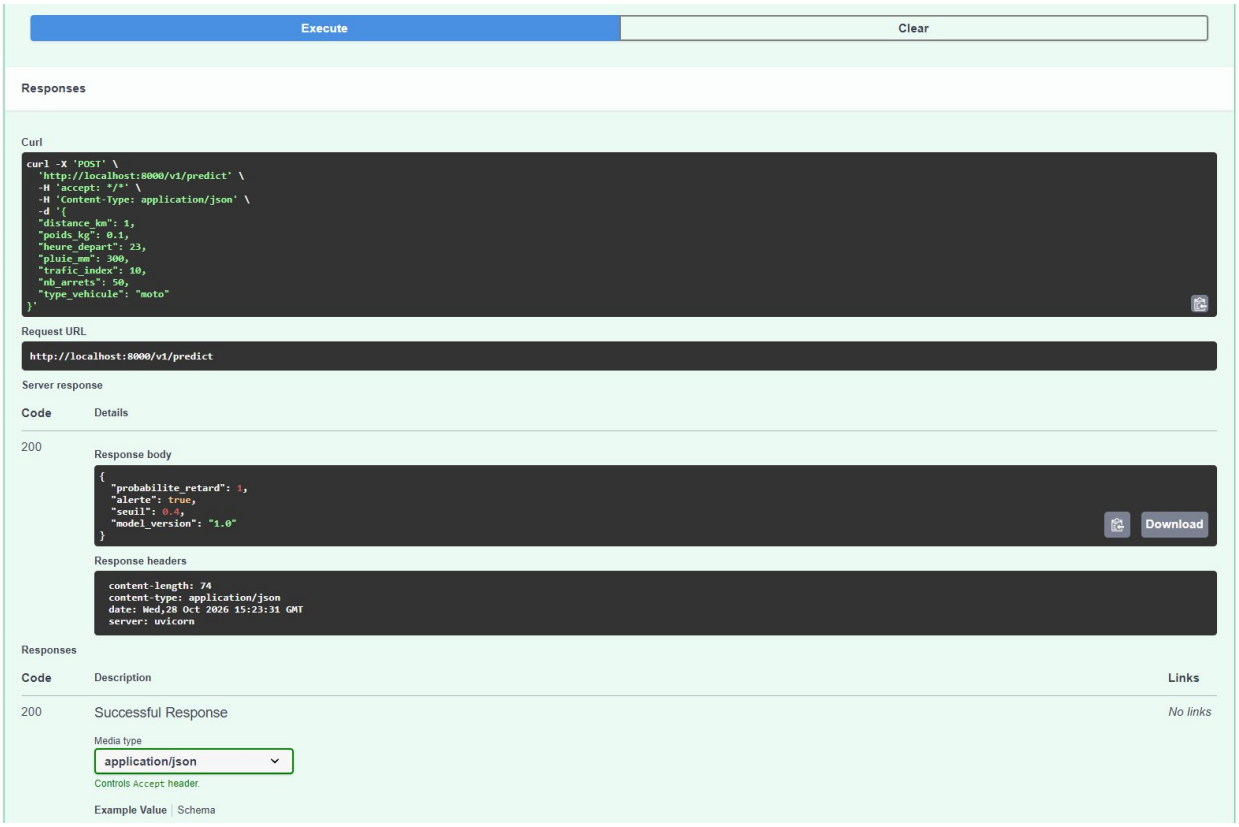
3. Implémentation de la fonction predict()

```python
@app.post("/v1/predict", response_model=PredictionOut)
def predict(livraison: Livraison):
    predictor = state.get("predictor")
    if predictor is None:
        raise HTTPException(
            status_code=503,
            detail="Modèle non chargé. Veuillez vérifier la présence du modèle ou ré-entraîner avec python -m logiai.train"
        )
    resultat = predictor.predict(livraison.model_dump())
    return PredictionOut(**resultat)

```

4. Vérification des quatre requêtes

Requête	Code HTTP	Explication
Livraison normale	200 OK	Données valides respectant l'ensemble du schéma et modèle présent en mémoire.
`distance_km = -5`	422 Unprocessable Entity	Valeur négative refusée par la borne `ge=1` vérifiée par Pydantic.
`type_vehicule = "avion"`	422 Unprocessable Entity	Valeur non conforme à l'énumération `Literal["moto", "fourgon", "camion"]`.
Modèle absent (dossier `models/` supprimé)	503 Service Unavailable	`state.get("predictor")` est `None` ; l'API lève explicitement une `HTTPException(503)`.



5. Questions de compréhension

1. **Pourquoi charge-t-on le modèle au démarrage et pas à chaque requête ?**
   Le chargement depuis le disque et la désérialisation du modèle (`joblib`) prennent des dizaines à centaines de millisecondes et sollicitent fortement le CPU/disque. Le charger une seule fois au démarrage (cycle `lifespan`) et le garder en mémoire RAM permet d'exécuter chaque prédiction en quasi-instantané (< 2 ms).
2. **Qui écrit le contrôle « pluie <= 100 » (ou 300), et où est-il écrit ?**
   Le développeur l'écrit dans `logiai/api/schemas.py` via le paramètre `le=...` de `Field()` . FastAPI et Pydantic interceptent et exécutent automatiquement cette validation avant tout traitement applicatif.
3. **Différence entre 422 et 503 ?**
   Le code **422** correspond à une erreur *client* (données envoyées erronées ou non conformes). Le code **503** correspond à une erreur *serveur* (le serveur est fonctionnel mais sa dépendance d'inférence est temporairement indisponible).
4. **Pourquoi une route `/v1/predict` plutôt que `/predict` ?**
   Le versionnement d'API garantit la rétrocompatibilité : on peut déployer plus tard une `/v2` sans impacter ni casser les clients existants qui continuent de consommer la version 1.
5. **Que se passerait-il si la distance était déclarée `int` ?**
   Toute distance contenant une valeur décimale (ex. `14.8` km) provoquerait une erreur de validation 422 ou une troncature forcée, privant l'API d'une précision kilométrique réelle.

6. Décision de conception : Modèle absent

Répondre 503 ou refuser de démarrer ? Qu'est-ce que chaque option fait perdre ?

- **Option A (Répondre 503) :** L'API démarre et l'endpoint `/health` indique `{"status": "ok", "model_loaded": false}`. Les dashboards de supervision peuvent identifier immédiatement la cause sans que le serveur ne tombe en panne complète. La documentation `/docs` reste également consultable. *Ce qu'on perd :* Les requêtes d'inférence échouent pour les utilisateurs.
- **Option B (Refuser de démarrer) :** Philosophie « fail-fast » où un orchestrateur (Kubernetes) ne dirigera aucun trafic vers un conteneur non opérationnel. *Ce qu'on perd :* Toute observabilité immédiate via `/health` ou `/docs`, provoquant des boucles de redémarrage (`CrashLoopBackOff`).

7. Tests et banc d'essai (Bloc 6)

Résultat après implémentation du schéma, de la route et des deux tests personnalisés :

```
================================== 15 passed, 75 warnings in 2.81s ===================================
PS C:\Users\PC\Downloads\logiai_starter> python -m pytest
========================================= test session starts =========================================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\PC\Downloads\logiai_starter
plugins: anyio-4.14.2, langsmith-0.10.13
collected 15 items

starter\tests\test_api.py ...............                                                    [100%]
```

Les deux tests ajoutés dans `tests/test_api.py` :

```python
def test_heure_24_refusee():
    with TestClient(app) as client:
        assert post(client, heure_depart=24).status_code == 422


def test_alerte_coherente_avec_le_seuil():
    difficile = dict(
        distance_km=479, poids_kg=2040, heure_depart=17,
        pluie_mm=46, trafic_index=10, nb_arrets=4, type_vehicule="camion"
    )
    facile = dict(
        distance_km=5, poids_kg=3, heure_depart=11,
        pluie_mm=0, trafic_index=1, nb_arrets=0, type_vehicule="moto"
    )
    with TestClient(app) as client:
        res_difficile = post(client, **difficile).json()
        res_facile = post(client, **facile).json()
        assert res_difficile["alerte"] is True
        assert res_facile["alerte"] is False

```

8. Client Streamlit (Bloc 7)







Gestion de l'API indisponible : Lorsque le serveur `uvicorn` est arrêté, le bloc `try / except` `requests.ConnectionError` intercepte l'erreur et affiche proprement dans l'interface : *« API injoignable : lancez uvicorn dans un autre terminal. »*

Question d'architecture : Ce client Streamlit n'importe plus `scikit-learn` ni le modèle. Cela découple totalement le frontend du backend d'IA. Une application mobile (iOS/Android) ou web en React n'a besoin que d'effectuer une requête HTTP POST standard, sans nécessiter d'environnement Python ni télécharger un modèle lourd.

9. Ticket de sortie

1. **Code pour pluie de 500 mm :** Code **422 Unprocessable Entity**, contrôlé par la validation Pydantic dans `schemas.py`.
2. **Chargement au démarrage :** Pour garantir la rapidité de réponse et éviter la surcharge disque à chaque requête.